想不到吧！開學第三週就可以開始做 final project！

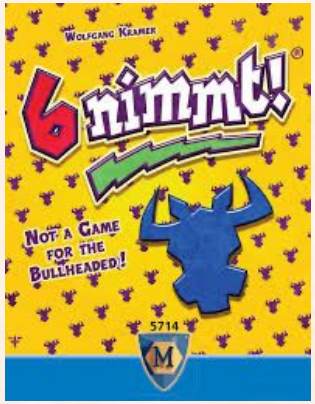
Code: https://github.com/yenshan0530/2026-FAI-Final-Release

Building a 6 Nimmt! Agent

FAI 2026 Final Project

Instructors: Shang-Tse Chen & Yun-Nung Chen

Due date: 2026.06.14. (Sun.)



FAQ and Updates

Game Introduction

The Objective: Stay Low!

Every card has "Bullheads" (🐮) on it. These are **Negative Points**.

Normal Cards

Ends in 5

% 10 == 0

% 11 == 0

The King

Winner = Player with the **FEWEST** Bullheads!

Game Setup

The Deck 104 cards (1 to 104). Each number is unique.

The Field **4 rows** are initialized, each with 1 starting card.

Player Hands Players receive **10 cards** each. Game lasts 10 rounds.

The Rounds In each round, cards are processed from **Smallest to Largest**.

How to Play: The Closest Fit

Cards must be placed in a row where the end-number is **smaller** than your card, but **closest** to it.

ROW 1

ROW 2

It **MUST** go to Row 2 (closest smaller number)

ROW 3

You Play

The Nightmare: The 6th Card

A row can only hold 5 cards. If you are forced to place the 6th card...

BOOM! You take all 5 cards (penalty += sum(3, 2, 5, 1, 3)),
and your "42" becomes the new start of the row.

What if your card is too small?

If your card is lower than **all** row-ends, you must take a row, then place your card.

Selection is AUTOMATED (No user choice)

ROW 1	42	49		2 🐮
ROW 2	37	39	41	3 🐮
ROW 3	98			1 🐮
ROW 4	61	63		2 🐮

You Play

Play with your friends to brainstorm strategies!

Play on BGA: https://boardgamearena.com/gamepanel?game=sechsnimmt

Or play with two decks of poker cards…?

Game Configurations

Game Specifications – Rules

- Recall: If your card is lower than **all** row-ends, you must take a row, then place your card.
- Which row will you take? We deterministically choose:
  - The row with the least penalty (number of bullheads)
  - If there is a tie, the row with less number of cards
  - If there is still a tie, the row with the smallest index

What you need to know about the game engine

Class engine:

- __init__(self, cfg, players) → setup game config, instantiate players, deal cards
- play_game(self) → play 10 rounds
- play_round(self) → request each player to play a card and additionally,
  - check for timeouts
  - check for disqualifications
  - check for invalid moves
  - check for memory usage
  - update game history

What you need to know about the game engine

Class engine:

- __init__(self, cfg, players) → setup game config, instantiate players, deal cards
- play_game(self) → play 10 rounds
- play_round(self) → request each player to play a card and additionally,
  - check for timeouts
  - check for disqualifications
  - check for invalid moves
  - check for memory usage
  - update game history

Game Specifications

- Timeouts: You have 1 second to play your card.
  - The timeout is a self-defined class

```
class TimeoutException(BaseException):
    pass
```

  - Once the time is out, we will send a TimeoutException to stop your player
    - We automatically play the smallest card for you
    - Your final penalty is your penalty + penalty gained by the min player
  - If time is out *and* you catch the exception
    - You get a penalty (see Game Specifications – Penalties)
    - Do not `except BaseException (as e):` or use a single `except:`
    - Be careful especially if you use LLM to write code!

Game Specifications

- Memory Errors:
  - Each player has a RAM usage limit of 1GB.
  - Be careful of storing huge dictionaries (while doing some tree search?)
- Multi-threading, Multi-processing:
  - Forbidden
- Invalid Moves:
  - If you play an invalid card (wrong type, number not in your hand, etc.), we automatically play the smallest card for you.

Game Specifications – Penalties

If you violate the following rules, your final score is deducted based on severity:

1. Modify the game history
   - We always pass a copied version of board history and your hands to you. No penalty, since you won't succeed anyway.
2. Change the random seed
   - We have a customized random number generator. No penalty, since you won't succeed anyway.

Game Specifications – Penalties

3. You purposely or accidentally use more than the RAM limit 1GB
   - If you hit this limit several times, your score for the final project will be multiplied by up to 50%
4. If you try to use multi-threading or multi-process
   - your score for the final project will be multiplied by up to 50%
5. Timeouts - If you exceed the time limit (1 second) when making decisions, we will automatically play the smallest card for you.
   - In the normal case, this would incur no additional penalty.
   - However, if you try to catch the exception, your score for the final project will be multiplied by up to 70%.
   - Be careful with torch—large batch sizes or models may cause timeouts.

Game Specifications – Penalties

If you run a tournament, these errors will be recorded and marked:

- DQ: Player timeouts and swallows a timeout exception.
- EXC: Player code raised an exception during their turn.
- OOM: Player's matchup subprocess was killed due to exceeding the max_memory_mb_per_matchup limit.
- ERR: Other errors occurred in the matchup.

Please refer to README.md in the project repository for more details.

Tournament Configurations

A Tournament

A tournament is what TAs would run to evaluate all of your players.

- A tournament consists of multiple partitions
  - A partition is dividing all players into groups of four
  - If there is a remainder, random players are padded
- Each partition consists of multiple games
  - Specifically, if there are N students, there would be ceil(N / 4) games
- We will run at least 500 partitions to mitigate randomness of final results
  - If we think the results haven’t converged, we will run more

Scoring Policy (Per Game)

- In each game, we rank players by their respective penalties
  - If there is a tie, all players receive the average of the total rank
    - For example, if Players 1-4 get penalty [10, 10, 10, 3],
      - They rank [(2+3+4)/3, (2+3+4)/3, (2+3+4)/3, 1] respectively
    - If players 1-4 get penalty [0, 1, 5, 5],
      - They rank [1, 2, 3.5, 3.5] respectively
  - If your player gets disqualified, they are replaced by a min player.

    Final penalty is your original penalty + penalty gained by the min player

Scoring Policy (Per Tournament)

- Your final ranking depends on your final average total rank.
- Conversion from the average total rank to your score for the final project is shown in Grading Policy.

Environment Specifications

Machine Specifications

- We will run the tournament on a machine at least as powerful as the CSIE workstations (ws1–ws7).
  - No GPU will be provided. Please use CPU for all computation.
  - Test your players on ws1–ws7 to check for OOM errors and timeouts.
- Remember:
  - Only 1 second for each decision
  - Only 1GB memory allowed for each player
  - You cannot use multi-process or multi-thread!

Environment Specifications

- Python version 3.13.11
- Allowed packages are listed in requirements.txt
  - Tournaments will be run under this setting.
- If you want to use any additional packages, email us for permission first
- It is recommended to use conda for environment settings
  - conda create -n FAI-final python=3.13.11
  - pip install -r requirements.txt
- Miniconda3 installation guide: https://www.anaconda.com/docs/getting-started/miniconda/install

Sample Code Explanation

File Structure Overview

Please refer to README.md in the project repository for more details.

- configs → game, tournament
- src → players/TA, __init__.py, engine.py, game_utils.py, tournament_runner.py
- .gitignore
- README.md
- requirements.txt
- run_single_game.py
- run_tournament.py

Write your own players

Please refer to README.md in the project repository for more details.

1. Create a **subdirectory** under `src/players/` (e.g., `src/players/student_id(lowercase)/`).
2. Add your player Python file(s) inside that directory.
3. Your player class must implement the `action` method.

Example: File: `src/players/student_id(lowercase)/best_player1.py`

```python
class BestPlayer1:
    def __init__(self, player_idx):
        self.player_idx = player_idx

    def action(self, hand, history):
        # hand: list of integers (your cards)
        # history: dict containing board state and past moves
        return hand[0] # Your logic here
```

Write your own players

Please refer to README.md in the project repository for more details.

```
class BestPlayer1:
    def __init__(self, player_idx):
        self.player_idx = player_idx

    def action(self, hand, history):
        # hand: list of integers (your cards)
        # history: dict containing board state and past moves
        return hand[0] # Your logic here
```

Required function 1

Required function 2

Notes for __init__():

- The `player_idx` is given just for you to lookup info on the board_history.
- You can pass in other arguments if you want for testing, but remember to remove them or set default values in your final submitted version.

Write your own players

Please refer to README.md in the project repository for more details.

Notes about action():

- Hands are sorted! You don't have to sort them again.
- It is recommended to compute values you need from the board_history instead of storing them, since your values might be incorrect if timeouts occur.
- The following are included in the history state:

```
history_state = {
    "board": self.board,
    "scores": self.scores,
    "round": self.round,
    "history_matrix": self.history_matrix,
    "board_history": self.board_history,
    "score_history": self.score_history,
}
```

- "scores" → current score of each agent
- "history_matrix" → cards players played in each round
- "board_history" → cards on the table in each round
- "score_history" → penalties players gained in each round

Running Games and Tournaments

1. Single Game Simulation

Use `run_single_game.py` to run a single match. It supports detailed logging and captures initial hands and history outputs.

```
python run_single_game.py --config configs/game/sample_single_game_config.json
```

2. Tournaments

Use `run_tournament.py` to run large-scale evaluations.

```
python run_tournament.py --config configs/tournament/sample_tournament_config.json
```

Optional config overrides: You can mix-and-match configurations via the command line using `--player-cfg`, `--engine-cfg`, or `--tournament-cfg`.

Saved Results – Game

```
output_data = {
    "config": game_config,
    "game_results": {
        "initial_hands": initial_hands,
        "final_scores": final_scores,
        "history": history
    }
}
```

```
full_history = {
    "board_history": self.board_history,
    "flags_matrix": self.flags_matrix,
    "final_scores": self.scores,
    "disqualified_players": list(self.disqualified_players),
    "timeout_counts": dict(self.timeout_counts),
    "exception_counts": dict(self.exception_counts)
}
```

Results are saved in `results/game`.

Saved Results – Tournament

```
output_data = {
    "config": config,
    "standings": final_standings,
    "history": history
}
```

```
matchup_history.append({
    "matchup_id": idx,
    "players": list(combo),
    "results": matchup_res_list
})
```

- Disqualification counts
- Timeout counts
- Exception catching counts
- Player total scores
- Player total ranks
- Player total games played
- Player ranks for each game
- Player scores for each game

Setting Configurations

Check configs/ in the project repository for examples.

You can set the configurations of a single game or a whole tournament.

- Player config
- Engine config
- Tournament config

There are many arguments for each configuration category.
Check the README in the project repository.

Setting Configurations – Players

Check configs/ in the project repository for examples.

All of these are valid formats:

```
"players": [
        {
            "path": "src.players.your_student_id.best_player1",
            "class": "BestPlayer1",
            "args": {}
        },
        [ "src.players.your_student_id.best_player1", "BestPlayer1" ],
        [ "src.players.your_student_id.best_player1", "BestPlayer2", {} ],
]
```

Important!

Grading Policy & Schedule

Grading Policy

Performance (70 pts)

- Midterm Submission (15 pts)
  - Tournament with all students + 40 baselines (a weaker subset)
- Final Submission (55 pts)
  - Tournament with all students + 55 baselines (the full set)

Report (30 pts)

Novelty: qualitative + quantitative, up to +5 pts, total capped at 100

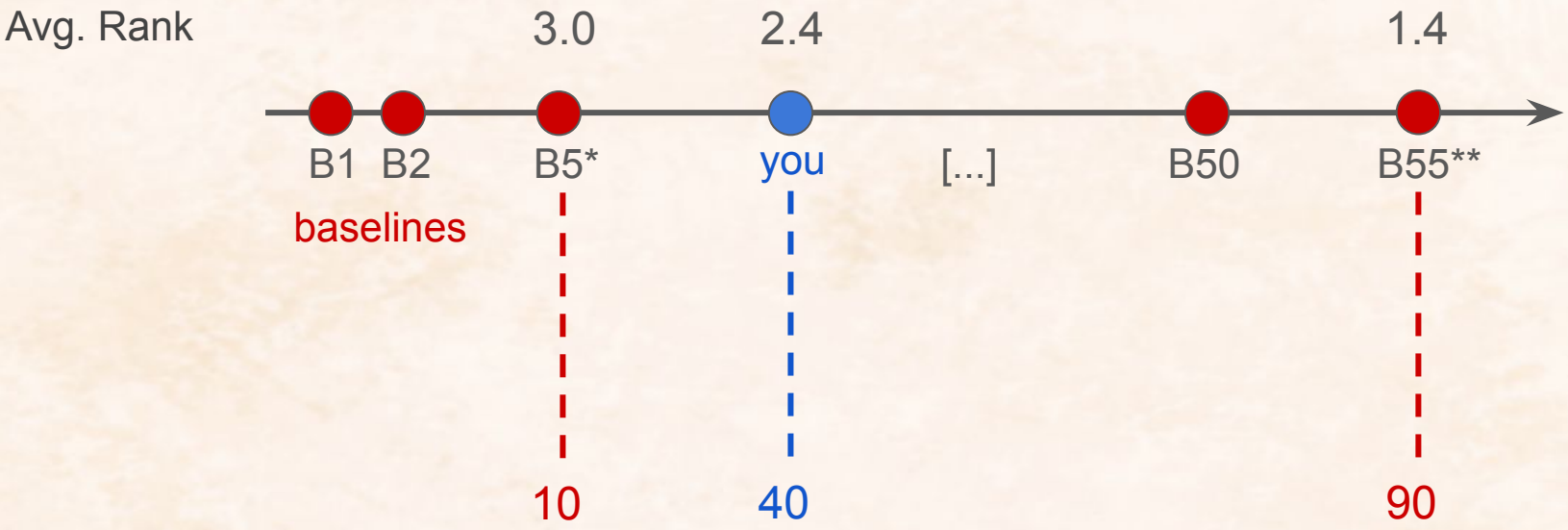
Baselines

Please refer to README.md in the project repository for more details.

- We will provide baselines in binary form.
- There are **55** baselines in total.
  - 5 baselines from B1-B27 will be released before the midterm.
  - 5 baselines from B1-B55 will be released after the midterm.
    - The released baselines are only a subset, so avoid overfitting to them.
- Baselines may be used for training, but **cannot** be submitted directly or imported into your final submission.

Baselines Scoring Criteria

Please refer to README.md in the project repository for more details.



For midterm, B5* = 60 pts, B20** = 90 pts
For final, B20* = 40 pts, B55** = 90 pts

Fun Activity - Guess your final rank!

You can submit a guess of your final rank in the tournament before the deadline.

- If you are the closest you may receive additional bonus points!
- If you guess your exact ranking, you may receive even more points!

Fun Awards!

TAs can also give bonus points.

For example,

- The best-performing non-model based solution
- TA's favorite algorithm
- Smallest submission size

Important: Early (Midterm) Submission

Since we are releasing a brand new game engine, there may be many bugs…

We need your help for early discovery of any potential issues!

There will be a midterm submission checkpoint (see next page).

- Please try to provide feedback before the midterm submission.
- After that, unless there are important bugs, we don't guarantee to fix the code.
- We will release the updated code before 5/20.

Timeline

3/9-3/15 Week 3	3/11 (Wed.): release baseline code
…	
4/20-4/26 Week 9	
4/27-5/3 Week 10	**4/29 (Wed.): midterm submission deadline**
5/4-5/10 Week 11	5/11 (Sun.) evaluate & release midterm submission results (we will release how many baselines you've beaten and a histogram of the class performance)
5/11-5/17 Week 12	
5/18-5/24 Week 13	5/20 (Wed.): release updated code (if necessary)
5/25-5/31 Week 14	
6/1-6/7 Week 15	
6/8-6/14 Week 16	**6/14 (Sun.): Code & report submission deadline**

Submission Policy

Code

You can submit two best players

- Name them:
  - BestPlayer1 in best_player1.py
  - BestPlayer2 in best_player2.py
- Try to think of diverse strategies that can counter different playing styles!
  - You don’t know what the player pool would be like.

Code

All files that were used to produce your final methods should be submitted

- e.g., training scripts, data collection pipelines, etc.
- If you have intermediate files (other than best players), you need to write an additional README.md explaining how to reproduce your results

Your code will be evaluated in a closed network, i.e., you cannot download models from the web.

Report

- Your report should include but not be limited to
  - **Methods tried:** Compare all the approaches you have explored.
  - **Core Idea:** Explain the main idea and logic of your 2 final algorithms.
  - **Implementation:** Details on optimizations and hyperparameter tuning.
  - **Self-Assessment:** What you are good at and what you struggled with.
  - **Strategic Profiling:** Opponent styles favorable and unfavorable to your 2 algorithms.
  - **Agent Comparison:** Compare the trade-offs of your two agents.
  - (Optional) Game analysis (vs. Texas Hold'em), future improvements...
- You should write your report in maximum **3 A4 pages in pdf format**
- Formatting: Use reasonable font sizes and margins.

Submission

- Please compress your player and the related files in a single .zip named with your student id in **lowercase**.
- The size of the submitted .zip file should be less than NTU COOL's submission restriction (~2GB).
- Don't hand in unnecessary pycache files.
- Same format for the midterm submission and the final code submission.
- Wrong format: -10% for the coding portion.

Submission

- Example:
  - bxx902xxx.zip
  - bxx902xxx/ ← Make sure there is a directory after decompression.
    - |-- README.md
    - |-- best_player1.py (with class BestPlayer1)
    - |-- best_player2.py (with class BestPlayer2)
    - |-- other related files needed
    - |-- Do not hand in __pycache__, MACOSX files!!!
      - Mac users: Use the command `zip -r student_id.zip student_id/` in your terminal.

Submission Portals

Midterm Submission
開放時間 3月 11 at 12:00 | 截止時間 04月29日 下午 11:59 | -/15 分

Final Submission (Code)
開放時間 3月 11 at 12:00 | 截止時間 06月14日 下午 11:59 | -/55 分

Final Submission (Report)
開放時間 3月 11 at 12:00 | 截止時間 06月14日 下午 11:59 | -/30 分

Guess your final rank!
開放時間 3月 11 at 12:00 | 截止時間 06月14日 下午 11:59

Other Policies

- Late submission policy: No late submission
- Plagiarism policy: Penalty up to 0 pts
  - Do not send your code to your friends
  - Discussion is encouraged
- LLM usage policy:
  - No restriction, but you should disclose any LLM usage in your report.
  - You are still responsible for what the LLM writes for you.
  - If your LLM writes a poorly written report or flawed code, the penalty is on you.

Contact Us

- Office hour: Wed 13:00-14:00 (R524)
  - Please email us before 23:59 the day before if you plan to come
  - Otherwise we don't guarantee to show up
- FAI TA email: fai-ta@csie.ntu.edu.tw
  - Please use the tag [Final Project] so we see your emails
  - If we don't reply your mail within 48 hours, please send a follow up
- Teaching assistants:
  - Yen-Shan Chen, Cheng-Yin Chang, Chih-Ho Chen, Jing Jin

Report Bugs and Issues

If you identify any bugs in the game engine or find the game rules ambiguous, please contact us as soon as possible.

- For minor clarifications or fixes, we will provide updates in the change logs of this slide deck.
- For major updates, we will make an announcement through NTU COOL.

NTU COOL Discussion Forum

We’ve created a discussion forum on NTU COOL for the final project.

This is mainly for you to discuss strategies and thoughts.

If you want to ask TA questions, please contact us through email.